Scenario 4: ALB and Kubernetes Ingress Routing Failure

Objective

Trace HTTP 404 responses across Route 53, ALB, Ingress, Service, endpoints, and pods.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/break-ingress.sh food-delivery
```

The script saves the current Ingress backend and changes the live route. Allow one or two minutes for the AWS Load Balancer Controller to reconcile before testing the public URL.

Confirm the failure from outside the cluster:

```
for i in {1..10}; do
  curl -s -o /dev/null -w '%{http_code}\n' \
    "https://food-delivery.$(cat .lab-domain)/"
done
```

Expected output: repeated HTTP `404` responses.

Situation

Users receive HTTP 404 from the food delivery home page even though the Kubernetes workloads report healthy.

Symptoms

- The public hostname returns HTTP 404.
- Kubernetes workloads remain Running and Ready.

What You Know

- `food-delivery.<lab-domain>` resolves and the ALB accepts connections.
- Pods are Running and Ready.

Start With the Symptom

1. Look at the body of the 404, not just the status code:

```
curl -s "https://food-delivery.$(cat .lab-domain)/"
```

Expected output: `Cannot GET /`. That is the Express backend's default 404 page, not the nginx frontend, which means public root traffic is reaching the backend instead of the frontend.

1. Compare it with an API path, which the backend is supposed to answer:

```
curl -s -o /dev/null -w '%{http_code}\n' "https://food-delivery.$(cat .lab-
domain)/api/restaurants"
```

Expected output: `200`. The ALB and backend work; only the route for `/` is wrong. Record the timestamp, then continue with Kubernetes and the AWS CLI.

Troubleshooting

1. Confirm that the workloads are healthy.

```
kubectl get pods -n food-delivery
```

Expected output: frontend and backend pods are Running and Ready. This moves the investigation away from pod health and toward request routing.

1. Inspect the public Ingress route.

```
kubectl describe ingress food-delivery -n food-delivery
```

Expected output: the `/` path points to `food-delivery-backend:4000`. It should point to `food-delivery-frontend:80`.

1. Confirm that both Services and endpoint sets exist.

```
kubectl get svc,endpoints -n food-delivery
```

Expected output: the frontend Service listens on port `80`, the backend listens on `4000`, and both have endpoints. This proves the failure is an incorrect Ingress destination rather than a missing Service or pod.

1. Confirm the ALB is present and serving the incorrect rule.

```
aws elbv2 describe-load-balancers --region us-east-1
aws elbv2 describe-listeners --load-balancer-arn <alb-arn> --region us-east-1
aws elbv2 describe-rules --listener-arn <listener-arn> --region us-east-1
```

Expected output: the ALB and HTTPS listener are active, and the host rule forwards traffic to the target group created from the incorrect backend Service. The Ingress backend selection is the root cause.

Important Clues

Follow DNS to ALB, listener, target group, Ingress, Service, endpoints, pod, and application. Determine why the backend answers a request intended for the frontend.

Root Cause

Record which routing hop sends traffic to the wrong destination before you apply the fix.

Fix

```
./scripts/chaos/break-ingress.sh food-delivery --undo
kubectl describe ingress food-delivery -n food-delivery
```

The source-of-truth manifest is `ingress/food-delivery-ingress.yaml`, whose backend is `food-delivery-frontend:80`.

Validation

Test `https://food-delivery.<lab-domain>/`, confirm the controller reconciles the Ingress, and verify that `/` returns `200` with the frontend page instead of `Cannot GET /`.

DevOps Lesson

Healthy pods do not guarantee healthy user traffic. Validate the whole routing path.

STAR Method

Situation

Summarize the user-visible 404 with healthy workloads.

Task

Explain the need to locate the failing routing hop.

Action

Describe the external, AWS, and Kubernetes traffic-path checks.

Result

State how external access was verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

Users received HTTP 404 responses from one of our applications even though every pod was healthy. My task was to locate the failing hop in the public request path. The 404 body came from the Express backend rather than nginx, and the Ingress revealed that `/` targeted port 4000 instead of the frontend on port 80. I corrected the route, waited for ALB reconciliation, and confirmed successful external requests.